

Demand Forecasting & Inventory Optimization Engine (Detailed Analysis)

This document provides an exhaustive theoretical background, comprehensive requirements gathering, and deep architectural system design for the **Demand Forecasting & Inventory Optimization Engine**.

1. Literature Review

1.1 The Role of Demand Forecasting in Supply Chain

Demand forecasting is the cornerstone of effective supply chain management. By predicting future customer demand accurately, businesses can align their procurement, manufacturing, and distribution activities. The primary goals are to:

- **Minimize Stockouts:** Ensure products are available when customers want them, preserving brand loyalty and revenue.
- **Reduce Holding Costs:** Avoid overstocking, which ties up capital and increases warehousing expenses.
- **Optimize Resource Allocation:** Improve labor scheduling and transportation logistics based on expected volumes.

1.2 Traditional Statistical Time-Series Analysis

Historically, organizations relied on moving averages and exponential smoothing. However, modern implementations utilize sophisticated statistical methods:

- **ARIMA (Auto-Regressive Integrated Moving Average):** Models the next step in the sequence as a linear function of the observations and residual errors at prior time steps. It excels in short-term forecasting when data is stationary.
- **SARIMA (Seasonal ARIMA):** Extends ARIMA by supporting univariate time series data with a seasonal component, making it ideal for retail products with yearly, monthly, or weekly cyclical patterns.
- **Holt-Winters Exponential Smoothing:** Captures level, trend, and seasonality by applying exponentially decreasing weights to older observations.

1.3 Advanced Machine Learning & Additive Models

The limitations of traditional models in handling exogenous variables (like price changes, promotions, or macroeconomic factors) led to the adoption of Machine Learning:

- **XGBoost (Extreme Gradient Boosting):** A highly efficient, scalable implementation of gradient boosted decision trees. By transforming time-series forecasting into a supervised regression problem (using lag features and rolling windows), XGBoost can capture complex, non-linear relationships and interactions between multiple features.
- **Facebook Prophet:** An open-source additive regression model designed for analyzing time-series data with strong seasonal effects (daily, weekly, yearly) and historical trend changes. Prophet is highly robust to missing data and shifts in the trend, handling

outliers gracefully, which is highly common in retail point-of-sale data.

1.4 Inventory Optimization Mathematical Models

Translating forecasts into actionable inventory decisions requires specific mathematical frameworks:

- **Safety Stock (\$SS\$):** The buffer maintained to mitigate risks of stockouts. Calculated using the formula: $SS = Z \times \sqrt{(\text{Lead Time} \times \sigma_D^2) + (\text{Average Demand}^2 \times \sigma_{LT}^2)}$ Where Z is the service level factor, σ_D is demand standard deviation, and σ_{LT} is lead time standard deviation.
- **Reorder Point (ROP):** The inventory level that triggers a replenishment order. $ROP = (\text{Average Daily Demand} \times \text{Average Lead Time}) + \text{Safety Stock}$
- **Economic Order Quantity (EOQ):** The ideal order quantity that minimizes total inventory costs (holding costs + ordering costs).

1.5 MLOps and Modern Serving Architectures

Deploying these algorithms requires transitioning from Jupyter Notebooks to robust engineering pipelines:

- **Model Registry & Tracking:** Using tools to track model experiments, hyperparameters, and artifacts (e.g., joblib serialization).
- **FastAPI:** A modern Python web framework used for building high-performance, asynchronous REST APIs to serve ML predictions with minimal latency.
- **Streamlit:** Facilitates rapid development of interactive web applications, democratizing data insights by allowing non-technical stakeholders to interact with complex ML models.

2. Requirements Gathering

2.1 Functional Requirements

- **Data Ingestion & Preprocessing:** The system must automatically read raw `.csv` or `.parquet` data, handle missing values, generate lag/rolling window features, and encode categorical variables.
- **Model Training Pipeline:** The system must allow users to trigger a training pipeline that evaluates Prophet, XGBoost, and SARIMA models side-by-side using a cross-validation split based on time.
- **Forecasting Engine:** The system must generate N -day horizon forecasts for specific SKUs (Stock Keeping Units).
- **Inventory Engine:** Based on user-defined Service Levels (e.g., 95%, 99%) and Lead Times, the system must calculate dynamic Safety Stock and Reorder Points.
- **API Serving:** Provide RESTful endpoints (e.g., `/api/v1/forecast/{sku}`, `/api/v1/inventory/{sku}`).
- **Interactive Dashboard:** Provide a Streamlit UI allowing users to upload new data, trigger forecasts, view historical vs. predicted plots, and download inventory recommendation reports.

2.2 Data Requirements

To function accurately, the system requires the following input schema:

- `date`: Datetime of the transaction.
- `item_id`: Unique identifier for the product/SKU.
- `store_id`: Location identifier (if multi-location).
- `sales_quantity`: Target variable.
- `price`: Continuous variable representing item cost.
- `is_promotion`: Boolean indicator of promotional events.
- `lead_time_days`: Average days it takes for a supplier to deliver the item.

2.3 User Stories & Acceptance Criteria

User Story 1: Demand Forecasting

- *As an Inventory Manager*, I want to see the predicted sales for the next 30 days for my top-selling SKUs, *so that* I can plan my purchasing budget.
- *Acceptance Criteria*: The dashboard displays a line chart with historical sales, predicted sales, and confidence intervals (upper/lower bounds).

User Story 2: Reorder Point Calculation

- *As a Procurement Officer*, I want to know the exact Reorder Point for each item, *so that* I place orders before running out of stock.
- *Acceptance Criteria*: The system calculates the ROP dynamically based on the 30-day forecast and outputs a tabular report highlighting items currently below their ROP.

User Story 3: Model Evaluation

- *As a Data Scientist*, I want to evaluate model metrics (RMSE, MAE, MAPE) on a hold-out test set, *so that* I can ensure the model is reliable before deploying it.
- *Acceptance Criteria*: The training pipeline generates an evaluation report and automatically selects the model with the lowest MAPE for API serving.

2.4 Non-Functional Requirements

- **Scalability**: The API must support horizontal scaling via Docker containers to handle bulk prediction requests.
 - **Performance**: End-to-end prediction via API should respond in under 200ms per SKU.
 - **Availability**: Target 99.9% uptime for the API endpoints via Azure deployment.
 - **Maintainability**: Code must adhere to PEP8 standards, utilizing modular design patterns for easy swapping of ML algorithms.
-

3. System Analysis & Design

3.1 Detailed System Architecture

The application is structured into discrete, highly cohesive micro-components:

1. Ingestion & Feature Store:

- Raw data is validated.
- Time-series features (day of week, month, seasonality indicators, lag features) are

engineered.

2. Model Training & Registry (MLOps):

- The `/src/models` directory contains abstract base classes for models to ensure uniform `fit()` and `predict()` signatures.
- Best-performing models are serialized using `joblib` and stored in a model registry.

3. Inference API (FastAPI):

- Loads the serialized models into memory on startup to minimize latency.
- Exposes asynchronous endpoints documented via Swagger UI (OpenAPI).

4. Client UI (Streamlit):

- Communicates exclusively with the FastAPI backend, acting as a decoupled presentation layer.

3.2 Entity Relationship Diagram (Conceptual Data Model)

```
erDiagram
    PRODUCT {
        string item_id PK
        string category
        float unit_cost
        int avg_lead_time
    }
    SALES_TRANSACTION {
        string transaction_id PK
        string item_id FK
        date date
        int quantity
        float price
        boolean promotion
    }
    FORECAST {
        string forecast_id PK
        string item_id FK
        date target_date
        float predicted_qty
        float lower_bound
        float upper_bound
        string model_used
    }
    INVENTORY_RECOMMENDATION {
        string item_id FK
        date generation_date
        float safety_stock
        float reorder_point
        int recommended_order_qty
    }

    PRODUCT ||--o{ SALES_TRANSACTION : "has historical"
```

```

PRODUCT ||--o{ FORECAST : "has predicted"
PRODUCT ||--o{ INVENTORY_RECOMMENDATION : "receives"

```

3.3 System Architecture Diagram

```

graph TD
    subgraph Client_Layer [Client Layer]
        UI[Streamlit Interactive Dashboard]
        ExtApp[External ERP / Supply Chain Software]
    end

    subgraph API_Gateway_Serving_Layer [API Gateway / Serving Layer]
        API[FastAPI Server]
        Swagger[OpenAPI / Swagger Docs]
        API --> Swagger
    end

    subgraph ML_Logic_Layer [ML & Logic Layer]
        Forecaster[Demand Forecaster Engine]
        InvOpt[Inventory Optimizer Engine]
        ModelReg[(Model Registry - Joblib files)]

        API --> Forecaster
        API --> InvOpt
        Forecaster --> ModelReg
    end

    subgraph Data_Pipeline_Layer [Data & Pipeline Layer]
        RawData[(Raw Data Storage)]
        PreProc[Feature Engineering Pipeline]
        ProcData[(Processed Data Store)]
        Trainer[Model Training Pipeline]

        RawData --> PreProc
        PreProc --> ProcData
        ProcData --> Trainer
        Trainer -->|Validates & Serializes| ModelReg
    end

    UI -->|HTTP GET/POST| API
    ExtApp -->|HTTP REST| API

```

3.4 API Endpoint Specifications (Swagger Outline)

- **GET /health**
 - **Description:** Checks if the API and models are loaded into memory.
 - **Response:** {"status": "healthy", "models_loaded": true}
- **POST /api/v1/train**
 - **Description:** Triggers the ML training pipeline on the latest data.
 - **Payload:** {"target_metric": "MAPE", "test_split_ratio": 0.2}

- **Response:** {"status": "training_started", "job_id": "12345"}
- **GET /api/v1/forecast/{item_id}**
 - **Description:** Returns the predicted demand for the next \$N\$ days.
 - **Query Params:** horizon=30
 - **Response:**

```
{
  "item_id": "SKU_100",
  "horizon": 30,
  "forecast": [
    {"date": "2026-05-01", "predicted_qty": 45, "upper": 52, "lower": 38},
    {"date": "2026-05-02", "predicted_qty": 42, "upper": 50, "lower": 35}
  ]
}
```
- **GET /api/v1/inventory/recommendations**
 - **Description:** Returns calculated ROP and Safety Stock based on the current active forecast.
 - **Query Params:** service_level=0.95
 - **Response:** Array containing safety_stock, reorder_point, and status (e.g., "OK", "REORDER_NOW") for requested items.

3.5 Infrastructure & Deployment Strategy

- **Containerization:** The FastAPI server and the Streamlit dashboard will be packaged in separate Docker containers using docker-compose.
- **Cloud Provider (Azure):**
 - Container Registry (ACR) to store Docker images.
 - Azure App Service or Azure Kubernetes Service (AKS) to host the containers.
- **CI/CD Pipeline:** GitHub Actions workflow triggered on pushes to the main branch. The pipeline will:
 1. Run pytest to execute unit tests on feature engineering and API endpoints.
 2. Build Docker images.
 3. Push images to Azure Container Registry.
 4. Trigger a rollout on Azure App Service.